

INF236 Oblig 3

Lars Haukland

May 2025

Abstract

Comparing algorithms for finding a *Maximal Independent Set* (MIS). A maximal independent set is an independent set where no vertex can be added while it still being an independent set. For this oblig I've implemented one sequential algorithm and one Random-selection parallel algorithm (LUBYS). Then I've ran these on large test graphs and compared the timing results and speedup. I also discuss a further alternative LUBYS algorithm to rather let vertices work with subgraphs and synchronize after some time.

1 Algorithm outline

1.1 Sequential MIS

This algorithm is the sequential algorithm taken from the [MIS wikipedia](#). This algorithm works by starting with an empty set I . Then adding some vertex v from $V(G)$ to I . Then remove v and all its neighbors from G . Repeat this until $V(G)$ is empty and we have a maximal independent set. This algorithm is actually quite fast as $MIS \in P$. The runtime is $O(n + m)$ because each vertex needs to be visited at least once with $+m$ from the adjacency list scanning.

1.2 Lubys algorithm

This algorithm starts by generating a random number for each vertex. And then the MIS will be choosed based on these values. The algorithm goes like this:

All threads gets a vertex v . Then they compare v 's value vs all neighbors values. If v dominates all neighbors, then v is included in the MIS. To maintain the independent rule all neighbors are then excluded from search queue. This repeats until there are no more vertices to search through.

Even tho this is pretty easy for me to explain I struggled alot with implementing. Mainly I struggled with finding a good way for threads to mark vertices as out of the search queue and then making this effective. I think my experience in c is what stops me here as I just really want to use Python sets or hashsets. This made it so that I implemented it in probably what is a little slower way. I work directly on the the global arrays marking vertices all over the place at the same time.

1.3 Alternative Lubys

One last algorithm I wanted to implement was an alternative implementation of lubys algorithm. This would instead of synchronizing at every step, rather synchronize after x steps. This might reduce overhead as long as all threads have some subgraph to work on. But because I implemented lubys so weirdly the first time i struggled to translate it into a way to do this. I'm not too sure if this would improve runtime/speedup but at least my intuition from oblig2 says that it should improve a little if implemented correctly.

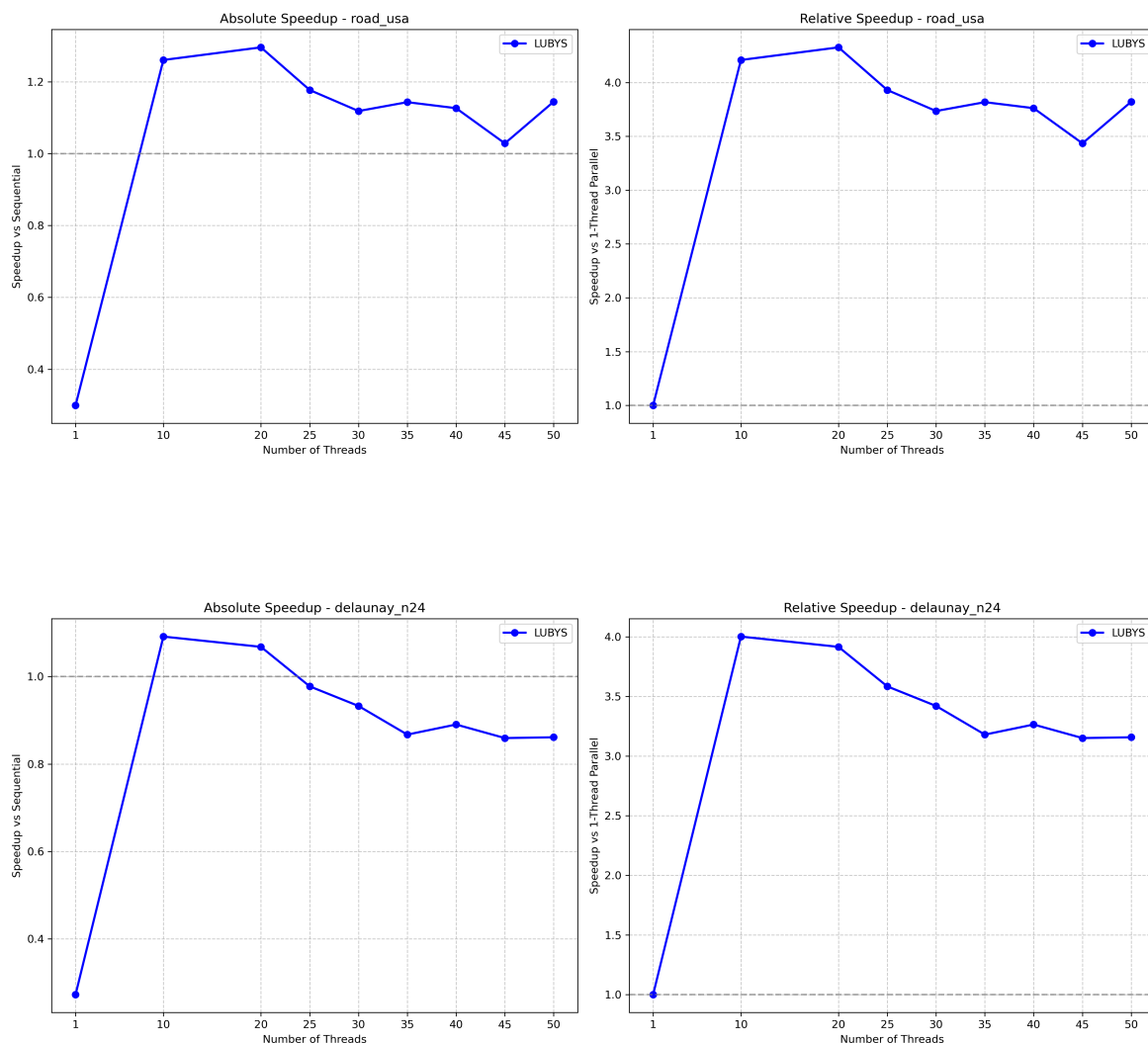
2 Experiments

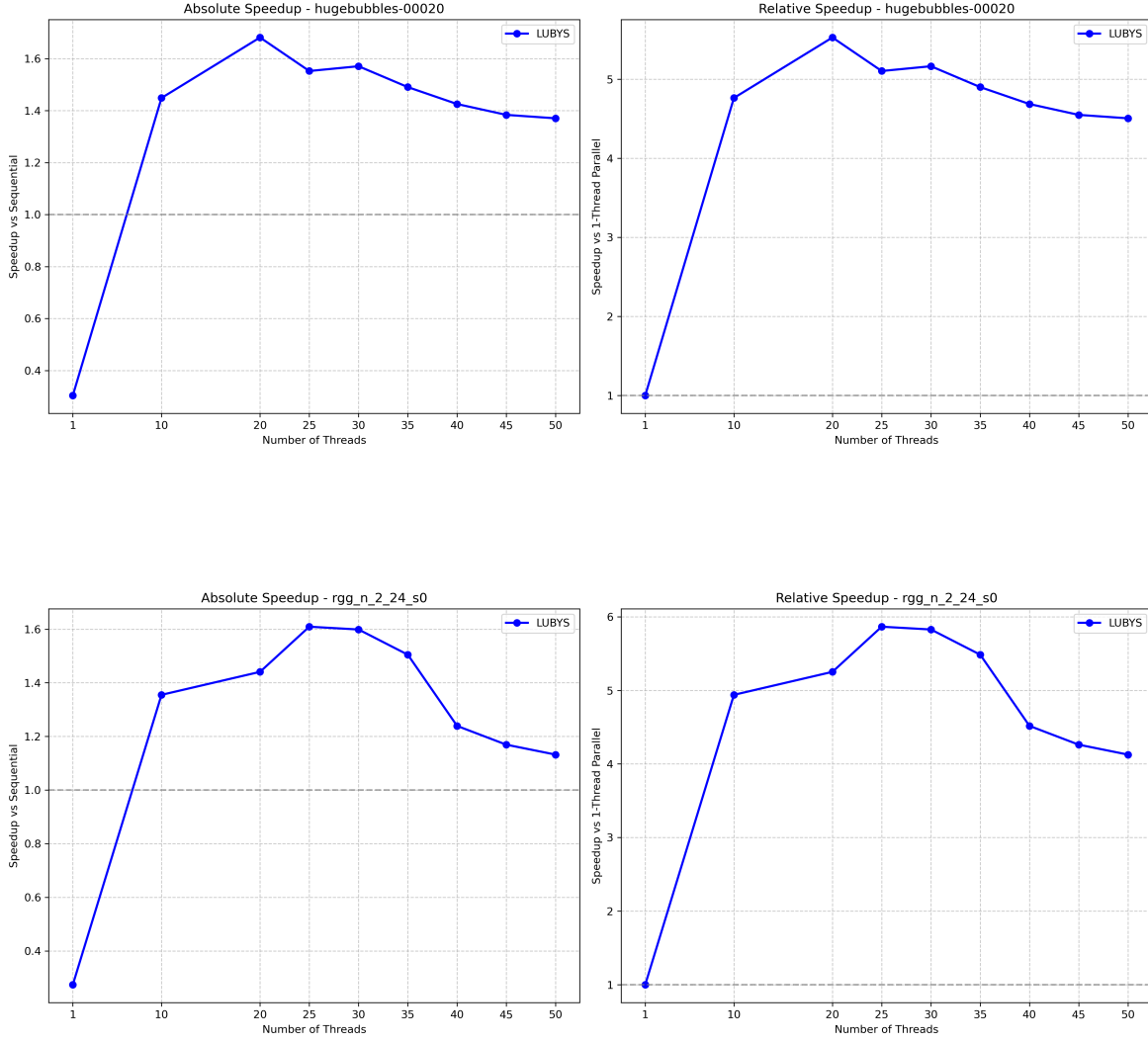
For the experiments I modified the driver.c file from oblig2 to run my algorithms. The specified configuration was to run 4 files (given in task), each 3 times (choosing the best result) and with 9 different

thread configurations. I kept my printing from last oblig to make the latex tables and the python script to make plots.

Below are plots for the lubys algorithm speedup. And maybe as I reacted it is not as impressive as I would have liked. As mentioned I think this was a rushed implementation but also hard to implement. I also did see some better numbers earlier this day but think alot of poeple are trying to run final experiments. I have some conclusions from these experiments

1. A better datastructure was needed to more efficiently handle a threads queue or search space. You can see my algorithm is pretty bad due to the speedup on sequential vs with 1 thread.
2. I think the only time lubys really needs many threads are in the first few iterations as the number of vertices to search pretty quickly decrease. This I think makes it sometimes more costly to have more threads.





3 Appendix

Table 1: Parallel LUBYS - road_usa ($|V| = 23947347$, $|E| = 28854312$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	0.733788	1.00	—
1	2.450543	0.30	1.00
10	0.582116	1.26	4.21
20	0.566319	1.30	4.33
25	0.623455	1.18	3.93
30	0.656037	1.12	3.74
35	0.641862	1.14	3.82
40	0.651390	1.13	3.76
45	0.713429	1.03	3.43
50	0.641149	1.14	3.82

Table 2: Parallel LUBYS - delaunay_n24 ($|V| = 16777216$, $|E| = 50331601$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	0.531854	1.00	—
1	1.950494	0.27	1.00
10	0.487295	1.09	4.00
20	0.498011	1.07	3.92
25	0.544064	0.98	3.59
30	0.570197	0.93	3.42
35	0.613234	0.87	3.18
40	0.597369	0.89	3.27
45	0.618892	0.86	3.15
50	0.617608	0.86	3.16

Table 3: Parallel LUBYS - hugebubbles-00020 ($|V| = 21198119$, $|E| = 31790179$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	1.240490	1.00	—
1	4.078363	0.30	1.00
10	0.856273	1.45	4.76
20	0.737787	1.68	5.53
25	0.799110	1.55	5.10
30	0.789762	1.57	5.16
35	0.832289	1.49	4.90
40	0.870314	1.43	4.69
45	0.896534	1.38	4.55
50	0.905202	1.37	4.51

Table 4: Parallel LUBYS - rgg_n_2_22_s0 ($|V| = 4194304$, $|E| = 30359198$)

Threads	Time (s)	Speedup vs Sequential	Relative Speedup
Sequential	0.132739	1.00	—
1	0.483776	0.27	1.00
10	0.097954	1.36	4.94
20	0.092129	1.44	5.25
25	0.082488	1.61	5.86
30	0.083026	1.60	5.83
35	0.088210	1.50	5.48
40	0.107110	1.24	4.52
45	0.113524	1.17	4.26
50	0.117235	1.13	4.13